



Original Article

Robust network anomaly detection using ensemble learning approach and explainable artificial intelligence (XAI)

Mohammad Kazim Hooshmand^{a,b}, Manjaiah Doddaghatta Huchaiah^b,
Ahmad Reda Alzighaibi^c, Hasan Hashim^c, El-Sayed Atlam^{c,d,*}, Ibrahim Gad^d

^a Department of Computer Science, Kabul Education University, Kabul, Afghanistan

^b Department of Computer Science, Mangalore University, Mangalore, India

^c College of Computer Science and Engineering, Taibah University, Yanbu, Saudi Arabia

^d Department of Computer Science, Faculty of Science, Tanta University, Egypt

ARTICLE INFO

Keywords:

IDS
Network anomaly detection systems
Ensemble learning
XGBoost
SMOTE
Oversampling
NSL-KDD dataset
Explainable artificial intelligence (XAI)
Prediction

ABSTRACT

Intrusion Detection Systems, specifically Network Anomaly Detection Systems (NADSs) are vital tools in network security. The NADSs are affected by data imbalance issues in classifying minority classes. Also, designing an efficient detection framework is sought after to achieve a higher detection rate for minority classes, especially when utilizing ensemble learning methods. To solve the issue of imbalanced data, a hybrid method of sampling techniques is proposed. This imbalance processing tool integrates the Synthetic Minority Oversampling Technique (SMOTE) and the K-means clustering algorithm (SKM). SMOTE over-samples the minority class, and K-means is used to perform a cluster-based under-sampling. We use Denoising Autoencoder (DAE) to select the top 15 features to reduce data dimensionality based on their higher weights. For anomaly detection, the XGBoost algorithm is deployed and the SHapley Additive exPlanation (SHAP) approach is deployed to provide explanations of the proposed techniques. The performance of the SKM-XGB model is assessed using the NSL-KDD and UNSW-NB15 datasets. A comparative analysis and series of experiments were carried out using several ensemble models with multiple base classifiers. The experimental findings indicate that the model's detection rate for binary classification and multiclass classification using the UNSW-NB15 dataset is 99.01% and 97.49%, respectively. The model achieves a 99.37% detection rate for binary classification and a 99.22% detection rate for multiclass classification on the NSL-KDD dataset. We conducted a comparative analysis of various ensemble models with multiple base classifiers. The results indicate that SKM-XGB outperforms the other investigated models and outperforms the performance of state-of-the-art models.

1. Introduction

Cyber attacks are a frequent occurrence in today's digitally connected society. They are unauthorized access to organizational network's digital assets. Firewalls provide security to the network but have limited capability to analyze network packets to identify various malicious activities and block them. Network Intrusion Detection Systems (NIDS) protect network infrastructure by monitoring and analyzing network traffic data.

There are mainly two categories of NIDS based on their detection techniques. One is a misuse detection (signature-based) technique, and the other is anomaly-based detection technique [9]. The misuse detection technique has a signature database for known attacks, and it

compares the new traffic with those already known attacks' signatures. If there is any match, the alarm is generated for the attack. On the other hand, anomaly-based detection techniques distinguish attacks by identifying deviations in traffic from normal patterns. The misuse technique is fast in detecting known attacks and produces a lower false alarm rate. However, it requires frequent manual updates of the signature database. Anomaly-based techniques are good in identifying unknown attacks, but it retains a higher false positive rate.

In anomaly-based method, machine learning algorithms are employed extensively. Different machine learning (ML) algorithms such as support vector machine (SVM) [4,42], decision tree (DT) [20,8], and random forest (RF) [34,10] have been employed to classify normal and abnormal traffic. In the purview of IDS, many studies used a combina-

* Corresponding author.

E-mail address: kazimhooshmand@gmail.com (M.K. Hooshmand).

<https://doi.org/10.1016/j.aej.2024.03.041>

Received 12 November 2023; Accepted 16 March 2024

Available online 26 March 2024

1110-0168/© 2024 THE AUTHORS. Published by Elsevier BV on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

tion of classifiers, and it's assumed that an ensemble learner performs better than an individual classifier due to several justifications such as computational, statistical, and representational [30].

In the ensemble learning approach, several machine learning algorithms' outputs are combined together to obtain a better result. If weak classifiers with overall low performance but good performance for some particular class are put together, then theoretically, it's possible to result in a good performance by combining the results of more than one weak classifier. There are various ensemble methods, such as Stacking, Bagging, and Boosting. Stacking, combines different classifications by a meta-classifier [3], the base classifiers are trained with the whole set of training data, and then the meta-model is trained on the outputs of the base classifiers. Bagging refers to the approach in which the same classifier is trained on different subsets of the same data. Boosting refers to a family of algorithms with the ability to transform weak learners into strong learners.

The Black-box nature of Artificial Intelligence (AI) algorithms makes it difficult for humans to understand, interpret, and sometimes even to accept the results produced by the model [31,6]. In artificial intelligence applications where findings and decision-making processes are crucial, black-box AI models are unsuitable. To address this problem, researchers have developed a set of AI models that can be easily interpreted and comprehended by end users; this set of tools is known as Explainable Artificial Intelligence, or XAI [5]. A lot of XAI models have been used to solve black-box model interpretability and explainability challenges in many real applications such as healthcare, military, energy, finance, and industry [12].

Explainable AI (XAI) approaches for post-hoc interpretability have recently emerged, giving rise to an entirely new generation of cybersecurity initiatives that contain enhanced levels of explainability for the user to understand how they work [25,6]. The explainable AI (XAI) tool is used to interpret both the global explanation of the model's results and the local explanations of each prediction in terms of features contribution [38]. For example, in [38], the authors demonstrate how the Random Forest and the SHAP framework can be utilized to get an understanding about the attributes of their model that contribute the most to the many different kinds of attacks.

This paper exploits the advantages of imbalance processing and integrates the advantages of XAI alongside ensemble learners in Network Anomaly Detection Systems (NADS) through intensive comparative studies among various base learners and different combination methods. The main contributions are as follows:

- This paper presents a new approach to imbalance processing using SMOTE oversampling technique and a cluster-based under-sampling with the K-means algorithm (SKM).
- The outcome of this approach is to balance data with minimum loss of information and without increasing data size.
- This paper proposes an Explainable AI-based ensemble-based network anomaly detection model, SKM-XGB, which combines the class imbalance method SKM and XGBoost algorithm to identify malicious traffic in imbalanced network traffic data with a high detection rate.

The remaining parts of this paper are organized as follows. Section 2 carefully describes the related works. Section 3 provides the description of the datasets. Section 4 provides an explanation of the proposed method. Section 5 presents details on the experimental results and analysis. Finally, the conclusion and future work are summarized in Section 6

2. Related work

Feature selection is vital for machine learning-based IDSs in today's fast-growing network traffic data. By removing unnecessary features, machine learning algorithms will be trained faster with lower com-

putational cost, and ultimately the accuracy will be boosted [35,6]. Hence, we propose variable selection as an integral part of the network anomaly detection model in this study. Here is a summarized review of some of the most related literature.

Nour Moustafa et al. [26] developed a combined method for feature selection. The method works based on central points of attribute values, which is then followed by the association rule mining. First, the dataset was split into equal segments to reduce processing time; the output of the Central Point technique was given as input to association rule mining to select significant features. Classifiers such as Logistic Regression, Expectation-maximization clustering, and the Naive Bayes method were deployed in the decision engine. The model's performance was evaluated on NSL-KDD and UNSW-NB15 datasets. Authors in [28] used the chi-square variable selection technique and correntropy-variation.

In [20], the authors have employed information gain as a variable selection method prior to feeding the data into their machine learning-based system for forensic IoT botnet activities. In [14], the authors used big data and deep learning technologies for their proposed IDS. They used K-means homogeneity metric variable selection along with the Random Forest, Gradient Boosting Tree, and Deep Feed-Forward Neural Network for binary classification as well as for multiclass classification.

Recent works have shown that over-sampling techniques have achieved good results on data imbalance problems [15]. Hence, over-sampling algorithms are widely used in the purview of IDSs for the problem of data imbalance. Authors in the work [36] claimed that SMOTE oversampling technique could efficiently improve the accuracy of the system.

Recent works have proven that the ensemble learning approach performs well in the domain of NIDS. In ensemble learning methods, a collection of machine learning algorithms are utilized to improve classification performance that could be achieved by a single classifier [32]. The basic idea is to combine multiple classifiers to exploit each individual algorithm's strength to obtain a more robust classifier.

Authors in [21] proposed an ensemble learning-based method combining four different base classifiers. They have shown that the proposed method yields a more accurate result for an intrusion detection system. Authors in [13] proposed an ensemble learning-based IDS. Different base classifiers are trained with a distinct set of variables, and then the outputs are combined together for the final decision. The results show that the overall error rate was reduced.

In [18], the authors present a combination of meta-learning with co-training techniques. They compared their proposal with the previous works, which used individual classifiers. The results show the effectiveness of their approach. Authors in [16] used an ensemble learning technique for network traffic classification. The performance of seven ensemble learning algorithms based on Decision Tree (DT) was compared with respect to the accuracy, latency, and byte accuracy. They have shown that some of the ensemble algorithms overcome single DT issues in terms of byte accuracy and accuracy.

Meanwhile, they presented a new ensemble classifier that is able to exploit imbalanced populations in the data to achieve a faster classification. It shows that the accuracy is high and the latency is low. In contrast to the previous works, we will use an imbalance processing as an integral part of the model; also, we conduct a pre-classification hyper-parameter tuning for base classifiers as well as for the ensemble model through a random search cross-validation technique.

3. Dataset

The datasets used for the models' evaluation are NSL-KDD and UNSW-NB15.

3.1. UNSW-NB15

This dataset [2,27] was generated in 2015 by the University of South Wales.

Table 1
Class-wise data samples in NSL-KDD and UNSW-NB15 datasets.

Dataset	Class	Train-set	Test-set	Validation-set	Total
UNSW-NB15	Normal	1,559,255	436,755	222,751	2,218,761
	Generic	151,430	42,418	21,633	215,481
	Fuzzers	17,039	4,773	2,434	24,246
	Reconnaissance	9,830	2,753	1,404	13,987
	Analysis	1,881	527	269	2,677
	DoS	11,491	3,220	1,642	16,353
	Shellcode	1,061	298	152	1,511
	Exploits	31,291	8,764	4,470	44,525
	Backdoors	1,637	458	234	2,329
	Worms	123	34	17	174
	Total	1,785,038	500,000	255,006	2,540,044
NSL-KDD	Normal	54,150	15,168	7,736	77,054
	DoS	37,517	10,508	5,360	53,385
	R2L	2,634	738	377	3,749
	Probe	9,893	2,772	1,412	14,077
	U2R	177	50	25	252
	Total	104,371	29,236	14,910	148,517

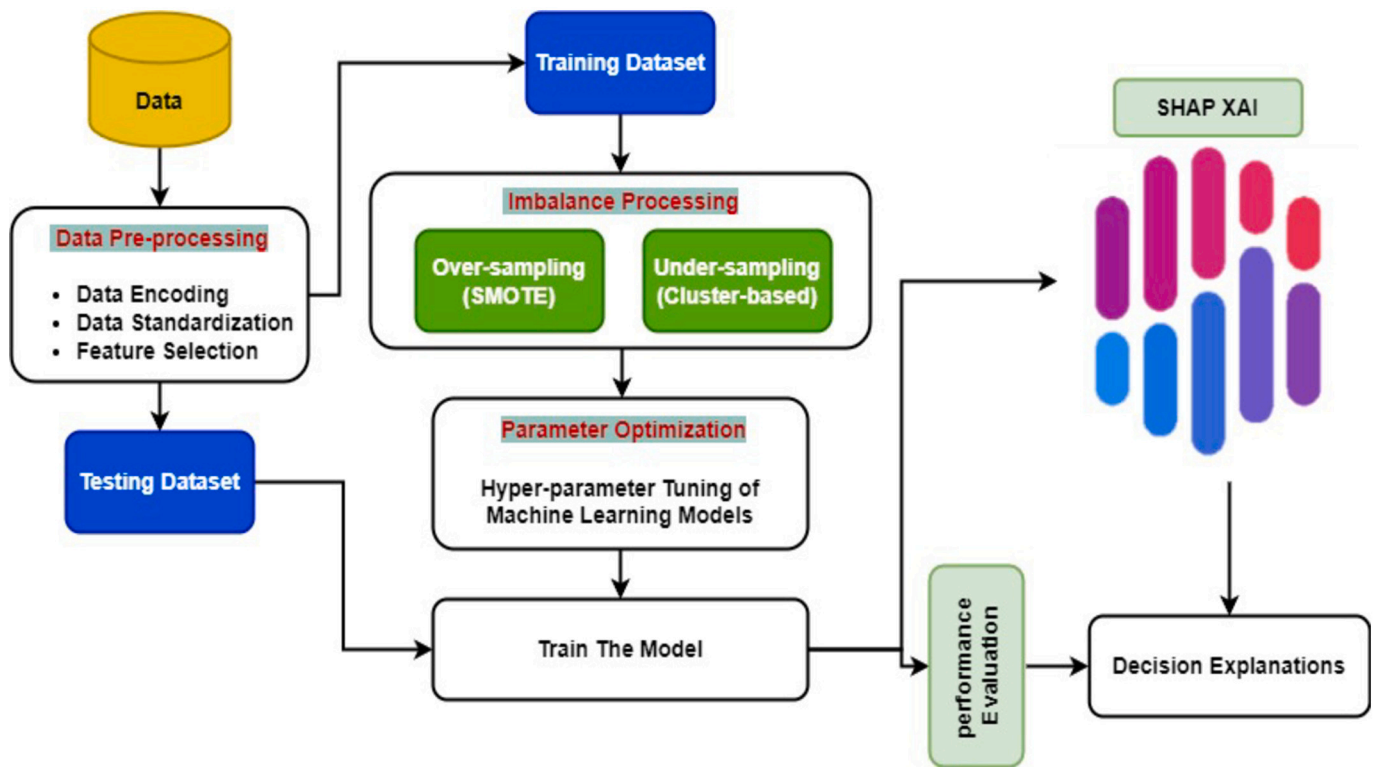


Fig. 1. The proposed method.

The researchers generated this dataset using three virtual servers and the Bro tool to collect 49 features. When compared to the KDD99 dataset, this dataset has more attacks and more features. It has nine types of attack and the normal class. UNSW-NB15 has been used in many recent works. We used the entire CSV files of the dataset in this study.

3.2. NSL-KDD

As the KDDCup99 has redundant and duplicate records, which impact on models' performance, the NSL-KDD is a refined version of KDDCup99 [19]. To avoid the redundancy issue, data samples are selected with higher caution while generating NSL-KDD. However, this dataset contains a large variety of files in various formats [22], we used KDDTrain(+.TXT) and KDDTest(+.TXT) files. NSL-KDD is one of the commonly used datasets in the IDS domain, and further details about it are given in [1]. The train-set, test-set, and validation-set counts for each class in both datasets are presented in Table 1.

4. The proposed method

The main steps of our methods are data preprocessing, imbalance processing, algorithms parameter optimization, training the model based on the best parameters, and evaluating the model (see Fig. 1).

4.1. Pre-processing step

Before the UNSW-NB15 data is preprocessed, the redundant attributes such as sport, stime, srcip, dsport, dstip, and ltime are removed [41]. The data preprocessing consists of data standardization/normalization, one-hot encoding, and feature selection. Both datasets have a few nominal features, the nominal features in the UNSW-NB15 are proto, service, and state. Similarly, the nominal features in the NSL-KDD are protocol type, flag, and service. The nominal features in both datasets have multiple different values which lead to generating many new features after applying one-hot encoding. After applying one-hot

Table 2
Selected features from NSL-KDD and UNSW-NB15.

Dataset	Selected features
UNSW-NB15	dtcpb, service_-, stcpb, dmeansz, sload, smeansz, dload, trans_depth, service_ftp-data, sttl, sloss, ct_ftp, djit, service_dns, ct_state_ttl
NSL-KDD	dst_host_srv_error_rate, service_http, protocol_type_udp, dst_host_same_srv_rate, duration, same_srv_rate, logged_in, srv_error_rate, dst_host_serror_rate, num_root, service_telnet, flag_SF, srv_count, dst_host_srv_serror_rate, service_other

encoding, the total number of features in UNSW-NB15 is increased to 202, whereas in NSL-KDD it reaches 121. In the next step of data preprocessing the equation (1) is applied to standardize the remaining features and then normalized based on Gaussian Distribution, with a mean of 0 and a variance of 1.

$$x' = \frac{x - \mu}{\sigma}, \quad (1)$$

where μ represents the mean and σ represents the standard deviation, and x' represents the normalized feature of the original variable.

The Gaussian Distribution, in its simplest basic form, is represented by the Equation (2)

$$f(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad (2)$$

where σ is the standard deviation, x represents the original feature, and μ represents the mean.

The DAE is deployed to select 15 features from each dataset. The selected features are given in Table 2.

4.2. Imbalance processing step

According to the Table 1, the UNSW-NB15 has relatively few records for several classes, including *backdoors*, *shellcode*, and *worms*. Similarly, the number of samples is small in NSL-KDD for classes such as *R2L* and *U2R*. If some class has a small number of instances compared to another class, such data is called imbalanced data. The imbalanced data impact models' performance. To balance the data, sampling techniques are recommended. On the other hand, oversampling alone will increase the size of data and will lead to a computational cost increase, and undersampling alone will cause information loss by eliminating some informative samples. To overcome the issue of imbalanced data and to balance between pros and cons of both sampling approaches, we applied a combined method of sampling. Our sampling technique consists of an oversampling using SMOTE and undersampling techniques using a cluster-based method with the help of the k-means algorithm. SMOTE generates new data samples by picking random points between minority classes and their k-nearest neighbors. In the under-sampling technique using the cluster-based approach, we divide majority class data samples into ten clusters, then randomly choose the samples in such a way that the total of selected samples should be equal to the number which we oversampled the minority class. The method of imbalance processing SKM is described in the Fig. 2.

4.3. XGBoost

XGBoost gained popularity in recent years; it is a scalable machine learning system [11] for tree boosting. XGBoost added several refinements and optimizations to Gradient Boosting Machine (GBM) aimed to facilitate the algorithm with further scalability [33]. Some of the key advantages of the algorithm are: (a) it features split finding algorithms that process sparse data with the default direction of nodes, deals with weighted data with merge and pruning operations, and optimizes splitting threshold, (b) it adds a regularization unit to the loss function presented in GBM which facilitates in creating simpler and more generative ensembles. Additionally, XGBoost supports distributed platforms

such as Apache Hadoop that can be more efficient in computational cost. We used XGBoost for classification purpose in this work.

4.4. Explainable AI (XAI)

Explainable AI (XAI), also known as Explainable Machine Learning (XML) or Interpretable AI, is a type of artificial intelligence (AI) in which users are able to understand the decisions or future predictions made by the AI [40,37]. It is completely opposed to the "black box" principle in machine learning, which states that not even the programmers can explain how an artificial intelligence arrived at a certain conclusion. Putting the social right to explanation into practice could be possible with the use of XAI. Even though XAI is not required by any laws or regulations, its implementation should still be considered. For example, XAI may enhance the customer experience of a service or product by making people more confident that the AI can make sound judgments or good decisions. Thus, the purpose of XAI is to explain what has been done, what is being done now, and what will be done in the future, as well as reveal the knowledge on which these actions are based [31].

The significant achievement of machine learning has led to an increase in interest in many applications of artificial intelligence (AI) [29]. However, the existing inability of machines to explain their conclusions and actions to users limits the effectiveness of these systems. The Explainable AI (XAI) seeks to: 1) develop ML models that are easily interpretable while ensuring a high level of learning performance; and 2) allow individuals to entirely understand, appropriately trust, and properly control the next generation of artificially intelligent partners [17].

Since XGBoosts models typically have complex ensemble structures that are hard to understand, we investigate using XAI approaches to build post-hoc explanations of the proposed model in the subsequent process. In particular, we apply the state-of-the-art SHAP technique for achieving correct explanations of tree-based models in a way that requires just polynomial amounts of time. The SHAP method provides an explanation for a specific sample of the data in the shape of a vector of "feature significance scores." These values will show the strength and direction of each feature's effect on the decision the model makes for that particular sample.

Generally, when SHAP is provided with a particular output from the model, it generates an explanation in the form of SHAP values (vector of importance scores). This explanation will have the following form: $\phi(x) = [\phi_1, \dots, \phi_i, \dots, \phi_N]$, where ϕ_i represents the influence that x_i has on the observed output. Local accuracy is one of the important aspects of the SHAP method, which guarantees that the total of all SHAP values for a given instance is equivalent to the difference between the output of the model for that instance and a constant baseline, ϕ_0 . i.e.,

$$\sum_{i=1}^N \phi_i = y - \phi_0 \quad (3)$$

The baseline value, denoted as ϕ_0 in the SHAP, represents the average or predicted output of the model that is obtained during the training phase. This is the model's first output before the effect of any features are taken into account.

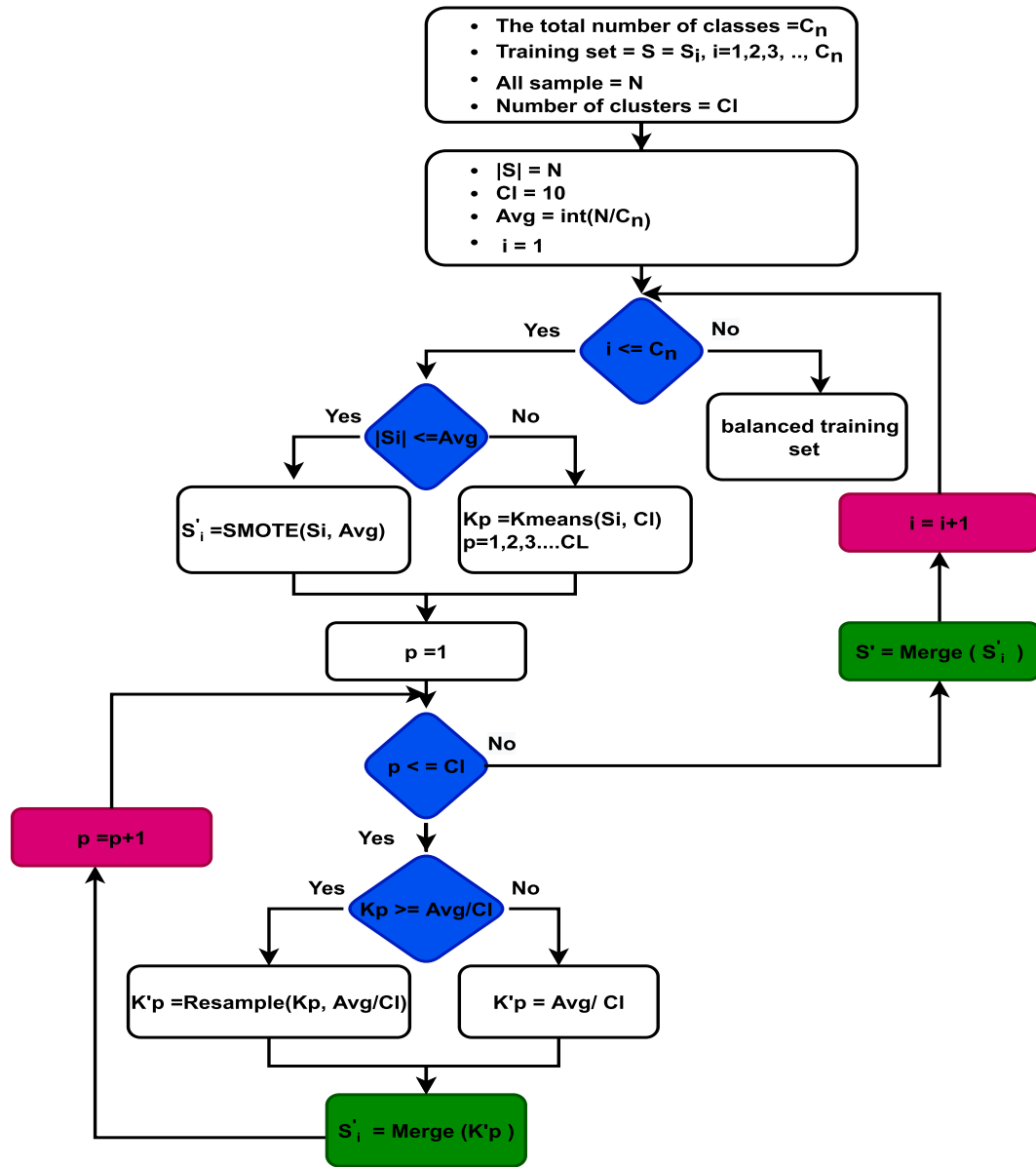


Fig. 2. The flowchart of the SKM.

Table 3
Experimental environment setup.

Parameter	Value
Programming Language	Python 3.6.13
OS	Windows 11 Pro
RAM	32 GB
CPU	Intel ® Xeon ® W-1250 @ 3.30 GHz

$$Accuracy = \frac{(TN + TP)}{(TP + FP + TN + FN)} \quad (4)$$

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

$$F1 = \frac{2 \times (Precision \times Recall)}{(Precision + Recall)} \quad (7)$$

$$FPR = \frac{FP}{(FP + TN)} \quad (8)$$

5. Experimental results and analysis

The proposed NAD was implemented using Python 3.6.13. The workstation details and configuration are provided in Table 3.

5.1. Evaluation metrics

The model's performance is measured using accuracy, recall, precision, F1, and false positive rate (or false alarm rate), which are calculated as:

5.2. Hyper-parameter optimization

We performed a parameter optimization for XGBoost and the other models used for comparison purposes in this work on both datasets. However, it requires much time and resources to test all combination possibilities of parameters with a larger range of values. We optimized three parameters of XGBoost using random search cross-validation and kept the remaining parameters with their default values. The optimized parameters of XGBoost are *n_estimator*, *learning_rate* and *max_depth*. To

Table 4
The parameters and best parameters of models.

Model	Parameters	Best parameters with UNSW-NB15	Best parameters with NSL-KDD
RF	max_features = ['auto', 'sqrt', None], n_estimators = np.linspace(10, 200), max_depth = np.linspace(3, 30), max_leaf_nodes = np.linspace(10, 50, 500), min_samples_split = [2, 5, 10], bootstrap = [True, False]	max_features = 'sqrt', n_estimators = 25, max_depth = 20, max_leaf_nodes = 38, min_samples_split = 10, bootstrap = False	max_features = None, n_estimators = 157, max_depth = 27, max_leaf_nodes = 48, min_samples_split = 5, bootstrap = True
KNN	weights = ['uniform', 'distance'], n_neighbors = [3, 5, 11, 19], p = [1, 2], algorithm = ['ball_tree', 'kd_tree', 'brute']	weights = 'uniform', n_neighbors = 19, p = 1, algorithm = 'brute'	weights = 'uniform', n_neighbors = 19, p = 1, algorithm = 'brute'
MLP	activation = ['tanh', 'relu'], hidden_layer_sizes = [(10,),(20,)], solver = ['sgd', 'adam'], alpha = [0.0001, 0.05], learning_rate = ['constant', 'adaptive']	activation = 'tanh', hidden_layer_sizes = (10,), solver = 'adam', alpha = 0.05, learning_rate = 'constant'	activation = 'relu', hidden_layer_sizes = (20,), solver = 'adam', alpha = 0.05, learning_rate = 'adaptive'
LGBM	learning_rate = [0.01, 0.05, 0.1, 0.15], n_estimators = [50, 100, 200, 300, 500], max_depth = np.arange(1, 7, 2)	learning_rate = 0.01, n_estimators = 300, max_depth = 5	learning_rate = 0.01, n_estimators = 300, max_depth = 5
AdaBoost	learning_rate = [(0.97 + x / 100) for x in range(0, 8)], n_estimators = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 20], algorithm = ['SAMME', 'SAMME.R']	learning_rate = 1.03, n_estimators = 3, algorithm = 'SAMME.R'	learning_rate = 1.03, n_estimators = 9, algorithm = 'SAMME.R'
XGB	learning_rate = [0.01, 0.05, 0.1, 0.15], n_estimators = [50, 100, 200, 300, 500], max_depth = np.arange(1, 7, 2)	learning_rate = 0.05, n_estimators = 500, max_depth = 5	learning_rate = 0.1, n_estimators = 500, max_depth = 5

Table 5
SKM-XGB binary classification performance on NSL-KDD and UNSW-NB15 datasets.

Model	Dataset	Accuracy %	Precision %	Recall %	F1 %	FAR %	Train_time (sec)	Test_time (sec)
SKM-XGB	UNSW-NB15	99.01	99.07	99.01	99.02	0.58	69	0.30
	NSL-KDD	99.37	99.37	99.37	99.37	0.64	16	0.04

compare our model's performance with the other ensemble methods, we use three other algorithms, such as RF, MLP, and KNN, as base classifiers with the Max-voting, Stacking, and Bagging combination methods. In addition, in order to demonstrate the efficacy of the proposed model, a comparison was conducted with ensemble models such as AdaBoost and LGBM. The same parameter optimization method with random search cross-validation is used to optimize the other models' parameters. Table 4 presents all the models, that are tested, the set of given parameters, and the selected best parameters with respect to both datasets.

5.3. Classification

We conducted binary classification as well as multiclass classification. Table 5 presents the results of binary classification for the SKM-XGB model on both datasets; Table 6 contains the result of the multiclass classification of SKM-XGB model as well as the results of other ensemble models which we tested for comparative analysis in this study. We compared our model with RF, MLP, KNN, AdaBoost, LGBM, Max-voting (with RF, MLP, KNN as the base classifier), Stacking (with RF, MLP, KNN as the base classifier), and Bagging (with RF, MLP, KNN as the base classifier) in term of multiclass classification.

From binary classification, we observe that SKM-XGB scores 99.01% and 99.37% detection rate for UNSW-NB15 and NSL-KDD, respectively, while the FAR are 0.58% and 0.64% respectively. In multiclass classification, SKM-XGB outperforms all the other ensemble models in all metrics such as accuracy, precision, recall, f-score, and FAR, while in the training time, Bagging with KNN is the best, and for the testing time, MLP is the best among all models on both datasets. Model's binary classification normalized and non-normalized confusion matrices on both datasets are presented in Fig. 3

Moreover, multiclass classification normalized and non-normalized confusion matrices on UNSW-NB15 and NSL-KDD are presented in the Figs. 4 and 5 respectively. We displayed a value on normalized confusion matrices if it is greater than 0.0001.

5.4. XAI explanation

The SKM-XGB classifier is used in the proposed framework because tree-based models outperform deep neural networks on tabular data in several applications [38]. Also, unlike deep learning models, tree-based models are naturally interpretable, which is significant for obtaining users' trust and enhancing the effectiveness of AI-based systems since it allows them to better understand the reasoning behind the results they were given. The Shapley [24] is adopted to extract local and global explanation for the proposed NAD model. The library works based on game theory method. SHAP-based model explanation measures the positive and negative contribution of all the features in the dataset by measuring the influence of each feature.

An ML model requires to not only be accurate but also easy to interpret. Machine learning models that are based on decision trees can be interpreted in a number of different ways, including based on the decision path, the information gain, and the heuristic value to features. In order to make the ML model more interpretable, the shap Python package is used to calculate the Shapley value as a heuristic value for each feature in the model. The best local explanation based on the precise computation of SHAP values for tree ensemble methods is implemented using TreeExplainer. As a result, the ML model's overall prediction can maintain local faithfulness. The extended property of local explanation can be used to capture the feature interaction provided by TreeExplainer for the tree-based ML model. Therefore, the TreeExplainer offers insightful data about the ML model's behavior.

Table 6
SKM-XGB multiclass classification performance on NSL-KDD and UNSW-NB15 datasets.

Dataset	Model	Acc. %	Precision %	Recall %	F1 %	FAR %	Train_time (s)	Test_time (s)
UNSW-NB15	RF	98.82	98.10	96.83	97.25	0.52	3.48	0.48
	MLP	98.85	97.95	96.37	96.96	0.23	48.22	0.17
	KNN	98.94	97.82	96.79	97.23	0.51	0.08	1882.16
	Voting_RF_MLP_KNN	98.84	98.07	96.68	97.17	0.52	55.24	966.18
	Stacking_RF_MLP_KNN	98.69	95.46	95.84	95.42	1.27	1265.88	825.59
	Bagging_RF	97.79	97.06	95.40	94.71	10.04	6.95	2.96
	Bagging_MLP	97.64	93.44	94.25	93.15	4.63	0.39	1.31
	Bagging_KNN	88.18	88.95	87.35	81.45	87.35	0.06	6.30
	AdaBoost	98.36	98.45	95.56	96.02	0.17	2.87	0.37
	LGBM	98.91	98.30	97.01	97.47	0.34	31.63	8.15
	SKM-XGB	99.08	98.46	97.49	97.84	0.61	721.72	3.41
NSL-KDD	RF	96.62	96.19	93.83	94.60	0.85	10.86	0.11
	MLP	96.87	96.31	94.33	94.98	0.94	63.48	0.01
	KNN	98.75	98.12	97.85	97.93	0.61	0.04	40.59
	Voting_RF_MLP_KNN	98.08	97.42	96.58	96.85	0.71	122.08	39.77
	Stacking_RF_MLP_KNN	98.47	97.67	97.41	97.49	0.71	527.36	37.65
	Bagging_RF	86.74	89.11	76.75	81.14	5.35	13.42	0.58
	Bagging_MLP	89.22	87.34	80.67	83.63	6.40	0.36	0.07
	Bagging_KNN	86.85	87.23	72.38	73.82	11.87	0.03	0.41
	AdaBoost	82.84	85.04	69.89	75.24	7.51	4.10	0.06
	LGBM	97.99	97.66	96.37	96.78	0.41	10.19	0.30
	SKM-XGB	99.57	99.26	99.22	99.24	0.20	190.61	0.16

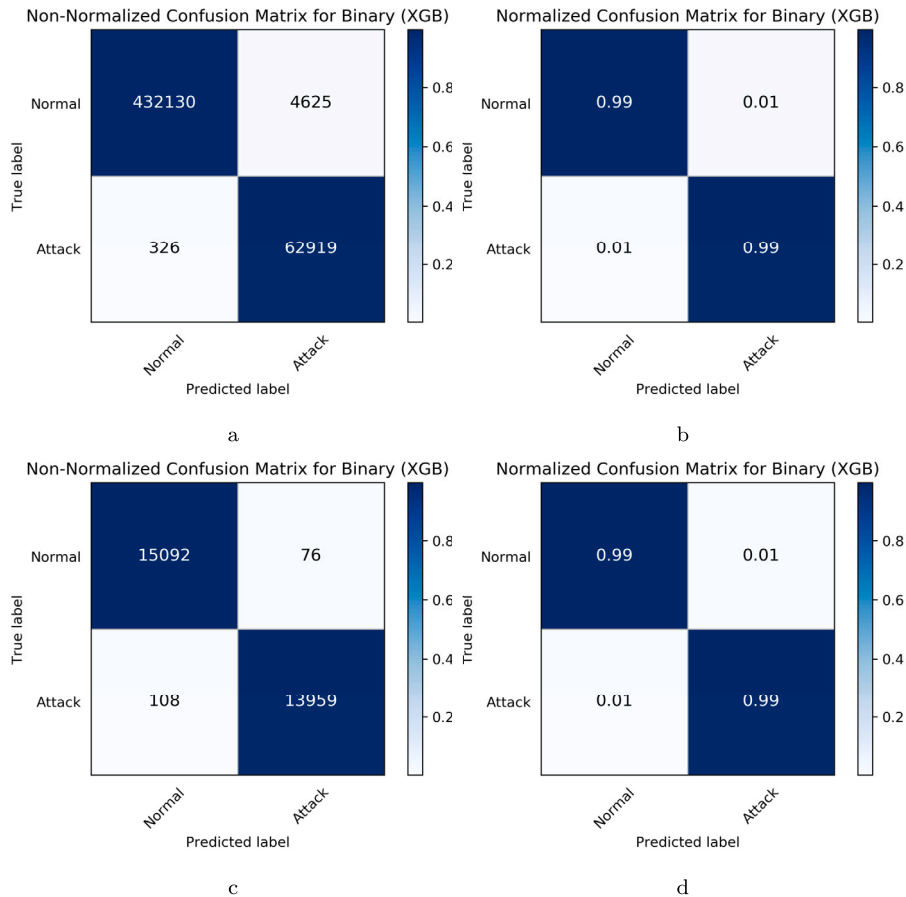


Fig. 3. Binary classification confusion matrix of SKM-XGB with UNSW-NB15 and NSL-KDD datasets (a) Non-normalized CM with UNSW-NB15, (b) Normalized CM with UNSW-NB15, (c) Non-normalized CM with NSL-KDD, and (d) Normalized CM with NSL-KDD, respectively.

The XGBoost model calculates the probability of a new instance. If the likelihood value is greater than 0.5, the instance under consideration is labeled as malware; otherwise, it is classified as benign. In addition, it provides a probability scale that illustrates the relative importance of the top 15. From the shape value, the probability values are obtained. For example, Fig. 6, shows an explanation of the output of XGB model and the contribution of the top 15 features in shap value. It

is clear to note that the “F 9” feature has more total model impact than the “F 13” feature, but for those samples where “F 13” matters it has more impact than “F 11”.

Fig. 7 illustrates the density scatter plot of SHAP values for each feature to determine how much impact each feature has on the model output for individuals in the validation dataset. Features are ranked by their total SHAP value magnitude, which is calculated by adding up all

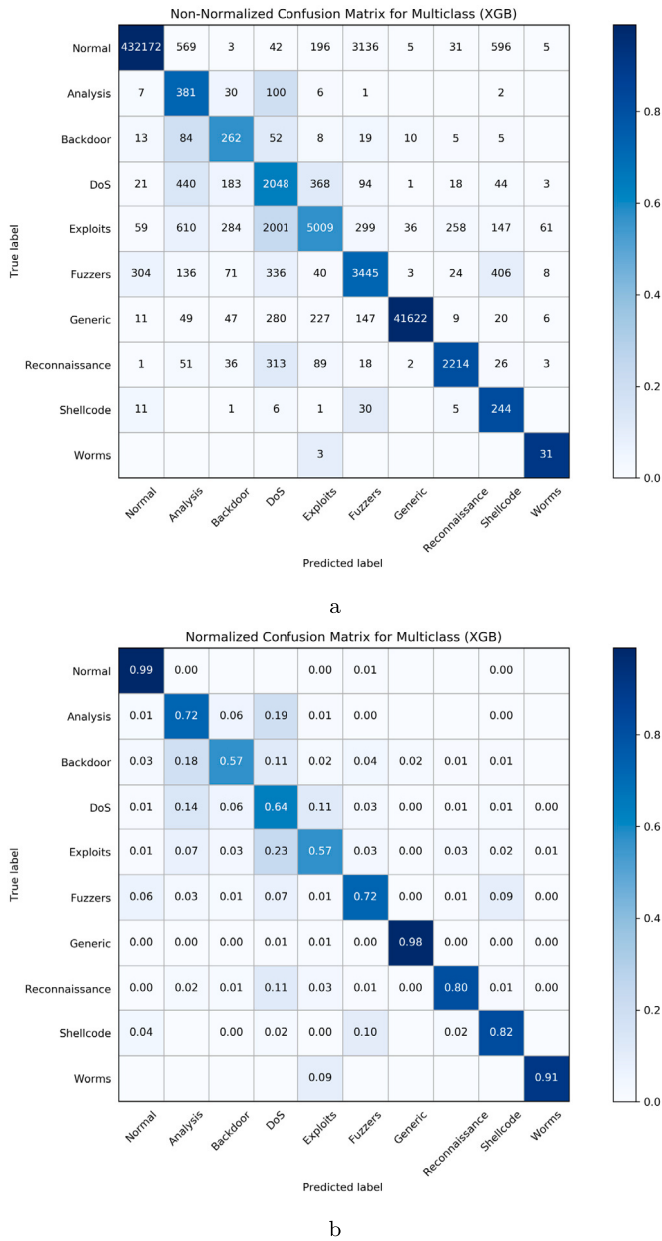


Fig. 4. Multiclass classification confusion matrix of SKM-XGB with UNSW-NB15 dataset (a) Non-normalized confusion matrix and (b) Normalized confusion matrix.

of the SHAP values in each sample. Features that have a positive impact on the decision of the model are shown by red bars in the visualization, while features that reduce the probability are shown by blue bars. For example, “F 13” has a huge influence that affects a small number of predictions by a large amount, but “F 2” has a little impact that affects all predictions by a smaller amount. Moreover, we see that other features, such as “F 2” or “F 8” also have negligible but positive effects the decision of the model, while others, like “F 1” and “F 6,” have noticeable but negative effects on the decision.

The impact of a single characteristic on the whole dataset is displayed through SHAP dependency visualizations. They plot the value of a feature against the SHAP value of that feature over several samples. In contrast to partial dependency plots, SHAP dependence plots only identify parts of the input space where there is data support, accounting for the interaction effects inherent in the features. In order to emphasize potential interactions, another feature is chosen for coloring at a single

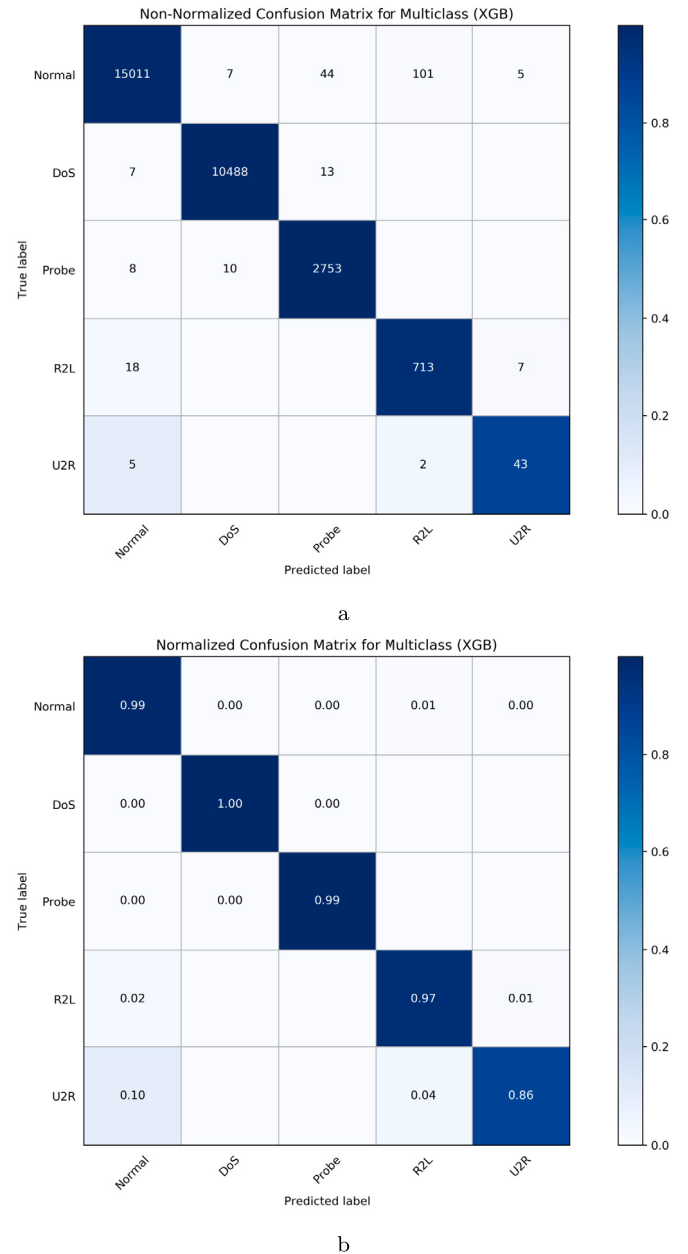


Fig. 5. Multiclass classification confusion matrix of SKM-XGB with NSL-KDD dataset (a) Non-normalized confusion matrix and (b) Normalized confusion matrix.

feature value where the vertical dispersion of SHAP values is caused by interaction effects.

Although the XGBoost model we trained above is extremely complex, we can observe the impact of varying a feature’s value on the model’s output by comparing the SHAP value for a feature to its actual value for all instances. The vertical dispersion of the data points shows how interaction terms affect a feature’s importance in relation to other terms. Fig. 8 shows a relationship between “F 0” and the target variable, and that relationship exhibits a roughly linear and positive trend. Furthermore, “F 0” regularly interacts with “F 1”.

5.5. Discussion

The experimental results show that our proposed method SKM-XGB significantly improved the detection rate and reduced the false positive rate (false alarm rate). However, the training time is slightly higher

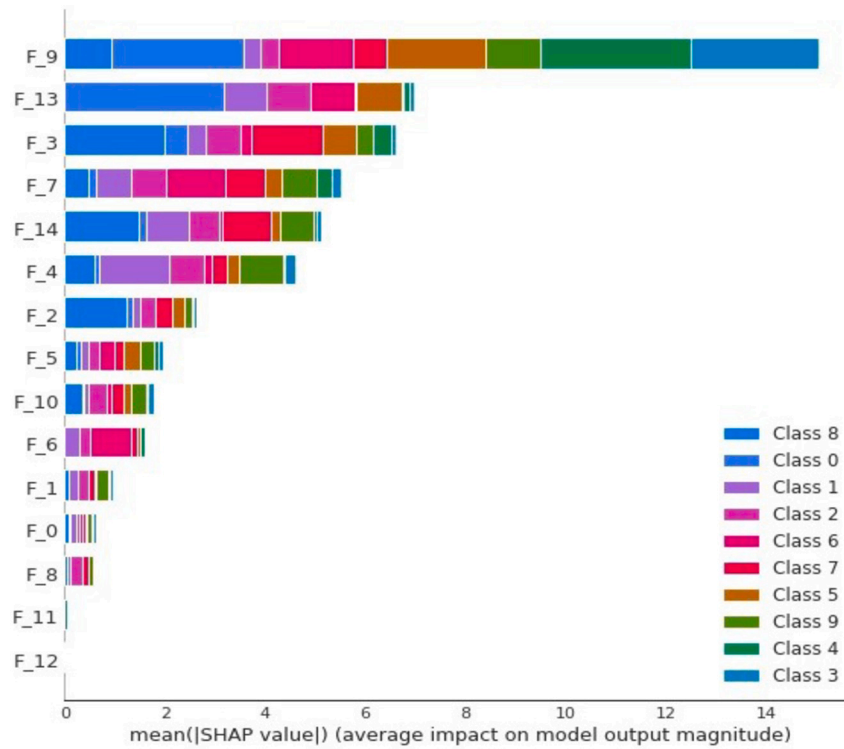


Fig. 6. The contribution of the top 15 features in bar plots of SHAP values for each category.

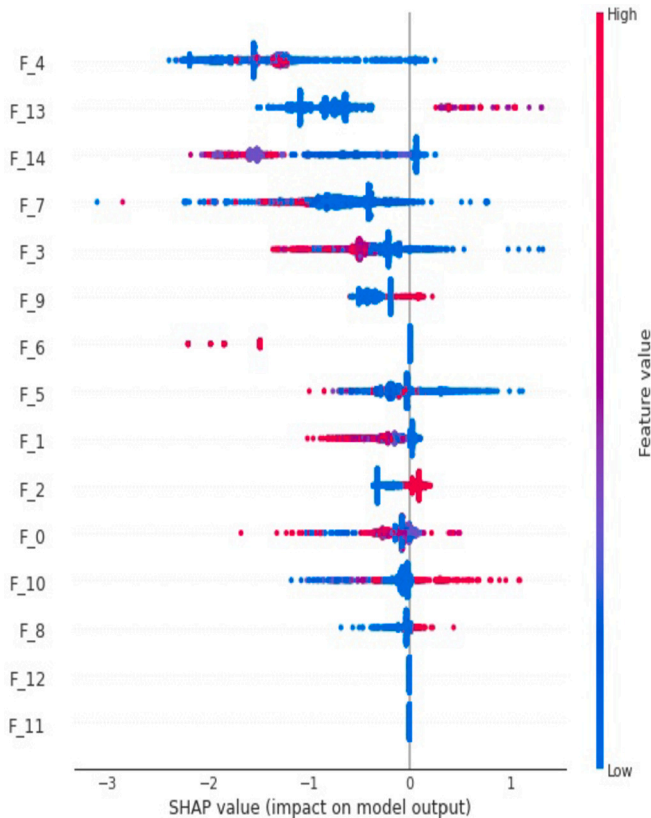


Fig. 7. The SHAP values for the XGB model.

than some of the models we compared, so our next studies will focus on implementing this model on a distributed platform such as Apache Spark, which is highly recommended for time efficiency, especially for XGB-based models. In Figs. 4 and 5 we can see that the recall is ef-

fectively boosted for the minority classes, which helps NADS to detect attacks with a small number of samples.

To show the efficacy and effectiveness of the SKM-XGB model, we compared it with the state-of-the-art intrusion detection methods, as given in Table 7. The table presents performance metrics for different datasets and problem types, along with various methods used for classification tasks. The datasets used are UNSW-NB15 and NSL-KDD, representing multiclass and binary problem types. The methods evaluated include RCNF, SCM3+RF, LightGBM+ADASYN, RepTree, IG+ANN, DAE+MLP, and the proposed model.

For the UNSW-NB15 dataset, in the multiclass problem type, RCNF achieved an accuracy of 95.98% and SCM3+RF achieved an accuracy of 95.87% with a detection rate (DR) of 97.40%. The proposed model outperformed both methods, achieving a remarkable accuracy of 99.08%, a DR of 97.49%, and an F1-score of 97.84%. The false alarm rate (FAR) for the proposed model was 0.61%.

In the binary problem type for the UNSW-NB15 dataset, RepTree achieved an accuracy of 88.95%, while IG+ANN achieved an accuracy of 97.04% with a FAR of 1.48%. DAE+MLP achieved an accuracy of 98.80% with a DR of 94.43% and an F1-score of 95.20%. The proposed model surpassed these methods, achieving an accuracy of 99.01%, a DR of 99.01%, and an F1-score of 99.02%. The FAR for the proposed model was 0.58%.

For the NSL-KDD dataset in the multiclass problem type, LightGBM+ADASYN achieved an accuracy of 92.57%. The proposed model demonstrated superior performance with an accuracy of 99.57%, a DR of 99.22%, and an F1-score of 99.24%.

In the binary problem type for the NSL-KDD dataset, RepTree achieved an accuracy of 89.85%. The proposed model outperformed RepTree with an accuracy of 99.37%, a DR of 99.37%, and an F1-score of 99.37%. The FAR for the proposed model was 0.64%.

Overall, the proposed model consistently achieved impressive performance across different datasets and problem types, showcasing its potential as an effective classification method.

Table 8 presents the overall performance of different models, including the proposed model, against benchmarks using the NSL-KDD

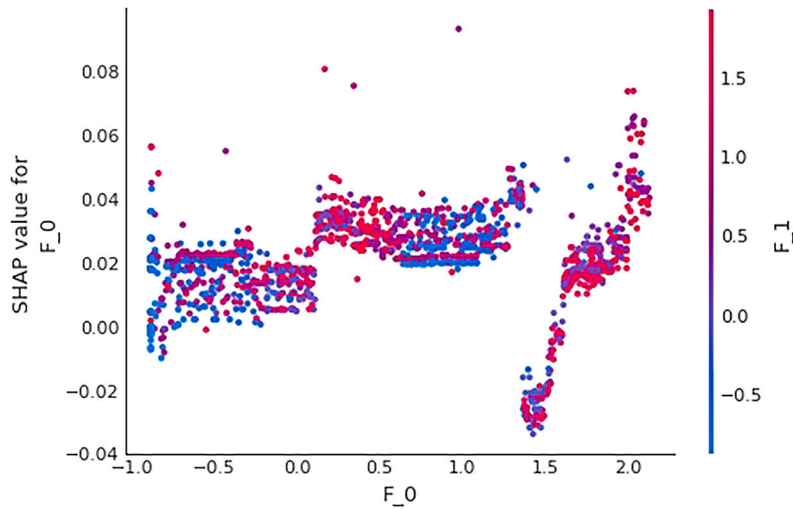


Fig. 8. The SHAP dependence plots for the XGB model.

Table 7

A comparison of the proposed model and the state-of-the-art approaches.

Dataset	Problem Type	Method	Acc. %	DR %	F1-score %	FAR %
UNSW-NB15	Multiclass	RCNF [28]	95.98	–	–	4.02
		SCM3+RF [10]	95.87	97.40	–	6.90
		LightGBM+ADASYN [23]	89.56	–	–	–
		The proposed model	99.08	97.49	97.84	0.61
	Binary	RepTree [7]	88.95	–	–	–
		IG+ANN [20]	97.04	–	–	1.48
		DAE+MLP [41]	98.80	94.43	95.20	0.57
		The proposed model	99.01	99.01	99.02	0.58
NSL-KDD	Multiclass	LightGBM+ADASYN [23]	92.57	–	–	–
		The proposed model	99.57	99.22	99.24	0.20
	Binary	RepTree [7]	89.85	–	–	–
		The proposed model	99.37	99.37	99.37	0.64

Table 8

Comparison results on the NSL-KDD dataset.

Model	Acc	Prec	Recall	XAI
SKM-XGB	99.57	99.26	99.22	Yes
XGBoost model [6]	93.28	91.05	97.81	Yes
Deep Learning [39]	80.6	82.8	80.6	Yes

dataset. The performance metrics evaluated are accuracy, precision, recall, and explainability (XAI). The proposed model outperformed the benchmarks, achieving an accuracy of 99.57%, precision of 99.2%, and recall of 99.57%. In comparison, the XGBoost model [6] achieved an accuracy of 93.28%, precision of 91.05%, and recall of 97.81%. Similar to the proposed model, the XGBoost model also demonstrated explainability. The Deep Learning model [39] achieved an accuracy of 80.6%, precision of 82.8%, and recall of 80.6%. Importantly, like the other models, the Deep Learning model also exhibited explainability. Additionally, the proposed model showcased superior performance in terms of accuracy, precision, and recall when compared to the benchmarks. Finally, these results highlight the efficacy and potential of the proposed model as a robust and interpretable solution in the context of the NSL-KDD dataset.

6. Conclusion

To address the problem of data imbalance without much loss of information, we proposed a hybrid approach of over-sampling using SMOTE and an under-sampling technique based on the K-means clustering method. The imbalance handling technique is then integrated with

an ensemble-based model using XGBoost for network anomaly detection that was trained on SHAP explanations describing the initial model's behavior during training. The proposed SKM-XGB performance is evaluated on two commonly used datasets, NSL-KDD and UNSW-NB15. According to the experimental results, SKM-XGB produces a detection rate of 99.01% and 97.49% for binary classification and multiclass classification on UNSW-NB15, respectively. For binary classification, the model has a detection rate of 99.37 percent on the NSL-KDD dataset, while for multiclass classification, it has a detection rate of 99.22 percent. Furthermore, the proposed SKM-XGB model significantly reduces the false alarm rate and provides an extra layer of explainability, which is critical for network anomaly detection systems. Comparing SKM-XGB with some previous works shows that the proposed model outperforms the state-of-the-art, which will be promising for future NADS.

Declaration of competing interest

The authors declare that there is no conflict of interest

Acknowledgements

The authors extend their appreciation to the “Deputyship for Research & Innovation, Ministry of Education in Saudi Arabia” for funding this research work through the project number: 445-9-343.

References

- [1] The NSL-KDD data set description, <https://www.unb.ca/cic/datasets/nsl.html>.
- [2] The UNSW-NB15 data set description, <https://www.unsw.adfa.edu.au/unsw-canberra-cyber/cybersecurity/ADFA-NB15-Datasets>.

- [3] A.A. Aburomman, M.B.I. Reaz, A novel SVM-kNN-PSO ensemble method for intrusion detection system, *Appl. Soft Comput.* 38 (2016) 360–372.
- [4] A.F.M. Agarap, A neural network architecture combining gated recurrent unit (GRU) and support vector machine (SVM) for intrusion detection in network traffic data, in: *Proceedings of the 2018 10th International Conference on Machine Learning and Computing*, 2018, pp. 26–30.
- [5] H.A. Alatwi, A. Aldweesh, Adversarial black-box attacks against network intrusion detection systems: a survey, in: *2021 IEEE World AI IoT Congress, AIoT, IEEE*, 2021.
- [6] P. Barnard, N. Marchetti, L.A. DaSilva, Robust network intrusion detection through explainable artificial intelligence (XAI), *IEEE Netw. Lett.* 4 (3) (2022) 167–171, <https://doi.org/10.1109/lnet.2022.3186589>.
- [7] M. Belouch, S. El Hadaj, M. Idhammad, A two-stage classifier approach using reptree algorithm for network intrusion detection, *Int. J. Adv. Comput. Sci. Appl.* 8 (6) (2017) 389–394.
- [8] T.T. Bhavani, M.K. Rao, A.M. Reddy, Network intrusion detection system using random forest and decision tree machine learning techniques, in: *First International Conference on Sustainable Technologies for Computational Intelligence*, Springer, 2020, pp. 637–643.
- [9] M.H. Bhuyan, D.K. Bhattacharya, J.K. Kalita, Network anomaly detection: methods, systems and tools, *IEEE Commun. Surv. Tutor.* 16 (1) (2013) 303–336.
- [10] A. Binbusayyis, T. Vaiyapuri, Identifying and benchmarking key features for cyber intrusion detection: an ensemble approach, *IEEE Access* 7 (2019) 106495–106513.
- [11] T. Chen, C. Guestrin, XGBoost: a scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [12] E. Dağlarlı, Explainable artificial intelligence (xAI) approaches and deep meta-learning models, in: *Advances and Applications in Deep Learning*, IntechOpen, 2020.
- [13] L. Didaci, G. Giacinto, F. Roli, Ensemble learning for intrusion detection in computer networks, in: *Workshop Machine Learning Methods Applications*, Siena, Italy, 2002.
- [14] O. Faker, E. Dogdu, Intrusion detection using big data and deep learning techniques, in: *Proceedings of the 2019 ACM Southeast conference*, 2019, pp. 86–93.
- [15] A. Fernández, S. García, F. Herrera, N.V. Chawla, Smote for learning from imbalanced data: progress and challenges, marking the 15-year anniversary, *J. Artif. Intell. Res.* 61 (2018) 863–905.
- [16] S.E. Gómez, B.C. Martínez, A.J. Sánchez-Esguevillas, L.H. Callejo, Ensemble network traffic classification: algorithm comparison and novel ensemble scheme proposal, *Comput. Netw.* 127 (2017) 68–80.
- [17] D. Gunning, D. Aha, DARPA's explainable artificial intelligence (XAI) program, *AI Mag.* 40 (2) (2019) 44–58, <https://doi.org/10.1609/aimag.v40i2.2850>.
- [18] H. He, X. Luo, F. Ma, C. Che, J. Wang, Network traffic classification based on ensemble learning and co-training, *Sci. China, Ser. F* 52 (2) (2009) 338–346.
- [19] G.E. Hinton, S. Osindero, Y.W. Teh, A fast learning algorithm for deep belief nets, *Neural Comput.* 18 (7) (2006) 1527–1554.
- [20] N. Koroniotis, N. Moustafa, E. Sitnikova, J. Slay, Towards developing network forensic mechanism for botnet activities in the IoT based on machine learning techniques, in: *International Conference on Mobile Networks and Management*, Springer, 2017, pp. 30–44.
- [21] J.R. Koza, R. Poli, A genetic programming tutorial, 2003.
- [22] S. Laqtib, K.E. Yassini, M.L. Hasnaoui, Evaluation of deep learning approaches for intrusion detection system in MANET, in: *The Proceedings of the Third International Conference on Smart City Applications*, Springer, 2019, pp. 986–998.
- [23] J. Liu, Y. Gao, F. Hu, A fast network intrusion detection system using adaptive synthetic oversampling and LightGBM, *Comput. Secur.* 106 (2021) 102289.
- [24] S.M. Lundberg, G. Erion, H. Chen, A. DeGrave, J.M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, S.I. Lee, From local explanations to global understanding with explainable AI for trees, *Nat. Mach. Intell.* 2 (1) (2020) 56–67, <https://doi.org/10.1038/s42256-019-0138-9>.
- [25] D.L. Marino, C.S. Wickramasinghe, M. Manic, An adversarial approach for explainable AI in intrusion detection systems, in: *IECON 2018 – 44th Annual Conference of the IEEE Industrial Electronics Society*, IEEE, 2018.
- [26] N. Moustafa, J. Slay, A hybrid feature selection for network intrusion detection systems: central points and association rules, in: *Australian Information Warfare Conference*, 2015.
- [27] N. Moustafa, J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: *2015 Military Communications and Information Systems Conference, MilCIS, IEEE*, 2015, pp. 1–6.
- [28] N. Moustafa, J. Slay, RCNF: real-time collaborative network forensic scheme for evidence analysis, *arXiv preprint, arXiv:1711.02824*, 2017.
- [29] Y. Pacheco, W. Sun, Adversarial machine learning: a comparative study on contemporary intrusion detection datasets, in: *Proceedings of the 7th International Conference on Information Systems Security and Privacy*, in: *SCITEPRESS – Science and Technology Publications*, 2021.
- [30] R. Polikar, Ensemble based systems in decision making, *IEEE Circuits Syst. Mag.* 6 (3) (2006) 21–45.
- [31] M. Ridley, Explainable artificial intelligence (XAI), *Inf. Technol. Libr.* 41 (2) (2022), <https://doi.org/10.6017/ital.v41i2.14683>.
- [32] J.W. Ryu, M. Kantardzic, C. Walgampaya, Ensemble classifier based on misclassified streaming data, in: *Proc. of the 10th IASTED Int. Conf. on Artificial Intelligence and Applications*, Austria, 2010, pp. 347–354.
- [33] O. Sagi, L. Rokach, Ensemble learning: a survey, *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* 8 (4) (2018) e1249.
- [34] P. Sangkatsanee, N. Wattanapongsakorn, C. Charnsripinyo, Practical real-time intrusion detection using machine learning approaches, *Comput. Commun.* 34 (18) (2011) 2227–2235.
- [35] M.A. Siddiqi, W. Pak, Optimizing filter-based feature selection method flow for intrusion detection system, *Electronics* 9 (12) (2020) 2114.
- [36] X. Tan, S. Su, Z. Huang, X. Guo, Z. Zuo, X. Sun, L. Li, Wireless sensor networks intrusion detection based on SMOTE and the random forest algorithm, *Sensors* 19 (1) (2019) 203.
- [37] M. Torky, I. Gad, A.E. Hassanien, Explainable AI model for recognizing financial crisis roots based on pigeon optimization and gradient boosting model, *Int. J. Comput. Intell. Syst.* 16 (1) (2023), <https://doi.org/10.1007/s44196-023-00222-9>.
- [38] S. Wali, I. Khan, Explainable AI and random forest based reliable intrusion detection system, *TechRxiv Preprint*, <https://doi.org/10.36227/techrxiv.17169080>, 2021.
- [39] M. Wang, K. Zheng, Y. Yang, X. Wang, An explainable machine learning framework for intrusion detection systems, *IEEE Access* 8 (2020) 73127–73141, <https://doi.org/10.1109/access.2020.2988359>.
- [40] T. Zebin, S. Rezvy, Y. Luo, An explainable AI-based intrusion detection system for DNS over HTTPS (DoH) attacks, *TechRxiv Preprint*, <https://doi.org/10.36227/techrxiv.17696972.v1>, 2022.
- [41] H. Zhang, C.Q. Wu, S. Gao, Z. Wang, Y. Xu, Y. Liu, An effective deep learning based scheme for network intrusion detection, in: *2018 24th International Conference on Pattern Recognition, ICPR, IEEE*, 2018, pp. 682–687.
- [42] W. Zong, Y.W. Chow, W. Susilo, Interactive three-dimensional visualization of network intrusion detection data for machine learning, *Future Gener. Comput. Syst.* 102 (2020) 292–306.